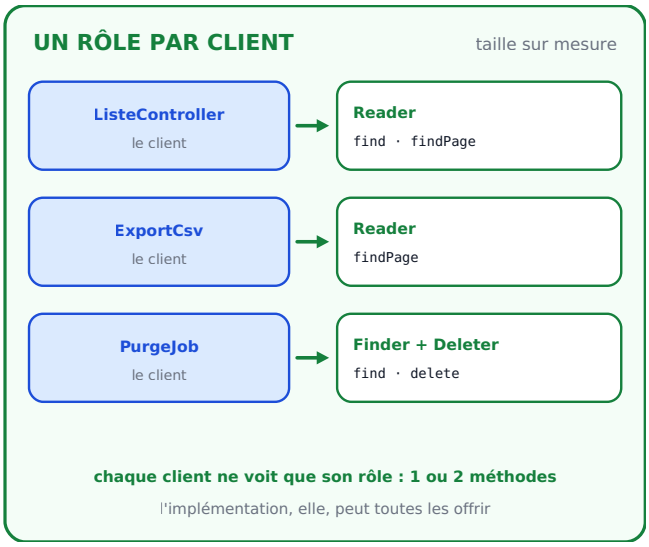
SOLID · LE I

Interface /Ségrégation

Un client ne dépend que de ce qu'il utilise

Tout le principe en schémas : la matrice clients × méthodes, le ratio d'utilisation, le faux de test minimal, et 10 cas réels découpés sous tes yeux.





1 Le principe en 4 règles 2
2 La matrice clients × méthodes 3
3 Le test ISP : ratio, faux minimal, lint 4
4 Anatomie de la violation · du respect 5-6
5 10 cas réels, avant / après en images + code 7-11
6 Le reste de la boîte en un coup d'œil 12
7 Les pièges : l'explosion d'interfaces 13
8 Arbre de décision · récap visuel 14-15
9 ISP dans SOLID · 6 refactorings 16-17
10 Exercices corrigés · fiche mémo 18-19

> Pour réviser : les pages 3 et 4 suffisent — la **matrice** (qui utilise quoi) et le **ratio** (utilisé / déclaré). Le reste n'est que déclinaison de ces deux mesures.

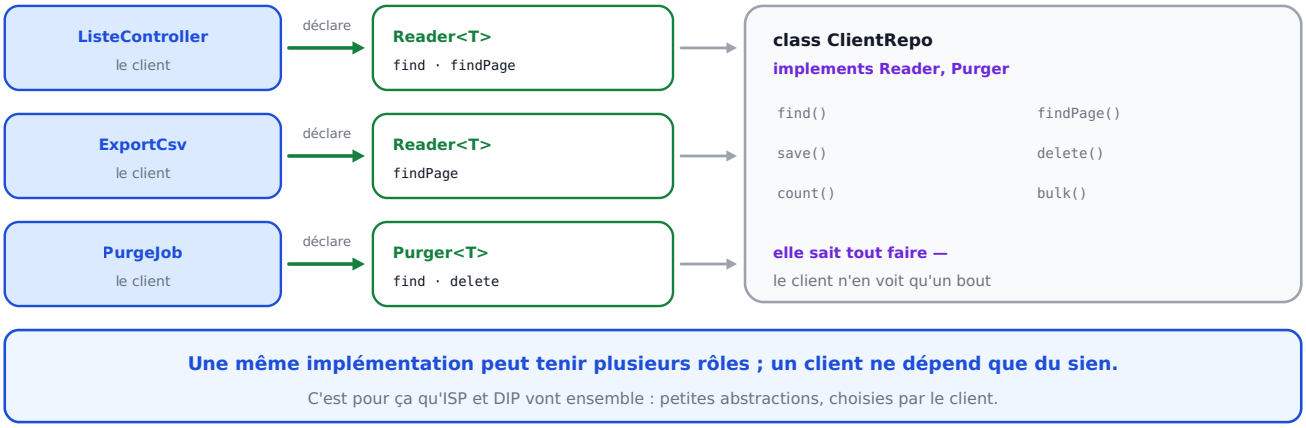
« Ne dépends que de ce que tu utilises. »
Interface Segregation Principle

1 · LE PRINCIPE

Une interface ne décrit pas un objet : elle décrit un besoin

« Aucun client ne doit être forcé de dépendre de méthodes qu'il n'utilise pas. » Le juge, c'est le client.

L'interface appartient au client, pas à l'objet



Une même implémentation peut tenir plusieurs rôles ; un client ne dépend que du sien.

C'est pour ça qu'ISP et DIP vont ensemble : petites abstractions, choisies par le client.

RÈGLE	FORMULATION COURTE	VIOLATION TYPIQUE
Par rôle	une interface par **rôle de client**	un `Repository<T>` de 8 méthodes pour un client qui lit
Minimale	le client déclare **le plus petit** contrat qui lui suffit	on injecte le service complet dans un module qui en veut un bout
Par capacité	on découpe par ce que l'objet **sait faire**	un `Animal` avec `voler()` + `nager()`
Par composition	on assemble au lieu de gonfler	une interface de 40 méthodes « pour éviter d'en créer trois »

AVANT — l'interface catalogue

```
interface Gestionnaire {
  open(): void
  save(): void
  export(): void
  sync(): void
}

class ArchiveZip implements Gestionnaire {
  open() { /* ok */ }
  save() { throw new Error("lecture seule") }
  export() { throw new Error("pas d'export") }
  sync() { /* rien */ }
}
```

APRÈS — une interface par rôle

```
interface Opener   { open(): void }
interface Saver    { save(): void }
interface Exporter { export(): void }

class ArchiveZip implements Opener {
  open() { /* ok */ }
}
class CloudDrive implements Opener, Saver, Exporter {
  // trois contrats, tous tenus
}

// le client : function lire(o: Opener) { … }
```

ON CROIT QUE…	EN RÉALITÉ
« ISP = les interfaces doivent être **petites** »	ISP = **ce client** ne doit hériter d'aucune dépendance inutile. La taille n'est qu'une conséquence.
« ISP = **une méthode par interface** »	c'est le zèle inverse : 8 interfaces `IFind`, `ISave`… sont illisibles (page 13).
« ISP ne concerne que les **interfaces** »	les **props** d'un composant, les **DTO**, les **imports** sont aussi des dépendances du client.
« il faut découper **toutes** les interfaces »	une interface que **tous** les clients utilisent en entier est honnête : on n'y touche pas.

Ce qu'ISP n'est pas : ce n'est pas « une méthode par interface », ni « les interfaces doivent être petites ». C'est : **le client ne doit rien hériter d'inutile**. Une interface de 10 méthodes est parfaite si un seul client les utilise toutes.

Qui utilise quoi ? La réponse tient dans un tableau

Une colonne que personne ne coche est une méthode en trop. Une ligne pleine de croix est un client qui dépend de tout.

La matrice clients × méthodes : qui utilise quoi ?

une colonne que personne ne coche est une méthode en trop ; une ligne pleine de croix est un client qui dépend de tout



Chaque paquet de colonnes est un rôle : c'est l'interface que le client devrait recevoir.

Après découpe : trois rôles, et l'implémentation qui les compose

interface Reader<T>
`find · findPage`
clients :
ListeController
ExportCsv
1 à 2 membres

interface Writer<T>
`save · delete`
clients :
Formulaire
PurgeJob
1 à 2 membres

interface Counter<T>
`count · bulk`
clients :
AdminPanel
1 à 2 membres

ClientRepo implémente les trois : personne ne dépend de ce qu'il n'utilise pas.

Le piège de la matrice : on la remplit souvent avec ce qu'on **croit** que les clients utilisent. Elle ne vaut que remplie avec le **code** : les appels réels, comptés.

Comment la remplir : trois minutes, un papier. Les lignes sont les **clients** (contrôleur, job, composant), les colonnes les **membres** de l'interface. Les paquets de colonnes qui apparaissent ensemble sont tes futures interfaces.

CE QUE MONTRE LA MATRICE	CONCLUSION
une ligne avec 1 ou 2 coches sur 8	ce client n'a pas besoin de cette interface : extraire son rôle
une colonne sans aucune coche	la méthode ne sert à personne : **la supprimer**
deux paquets de colonnes qui ne se croisent jamais	deux rôles distincts : **deux interfaces**
toutes les cases cochées	l'interface est honnête : ne pas la découper

Ce que fait un client, dans les faits

```
# quels membres chaque fichier client appelle-t-il ?
grep -rho "repo\.[a-z]*" src | sort | uniq -c

# 17 repo.findPage   ← le rôle Reader
#  2 repo.count      ← le rôle Counter
#  0 repo.bulkInsert ← méthode morte
```

Ce qu'on en déduit

```
// la matrice se fait sur le CODE, pas sur
// l'intention : on compte les appels

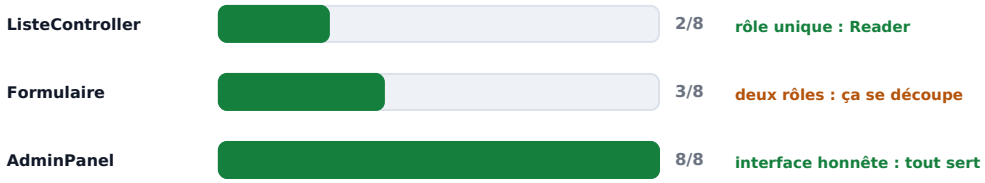
interface Reader<T> {              // 17 appels
  findPage(n: number, s: number): T[]
}
interface Counter {                // 2 appels
  count(): number
}
// bulkInsert n'apparaît nulle part → supprimée
```

3 · LE TEST

Mesurer au lieu de discuter

ISP est le principe le plus **mesurable** de SOLID : un ratio suffit.

Test 1 — le ratio, client par client



Un ratio sous 50 % veut dire que deux rôles sont mélangés.

Test 2 — le faux de test : s'il faut 8 méthodes, l'interface est fausse

```
// interface taillée :
class FauxReader implements Reader {
findById() { return null }
findPage() { return [] }
}
// interface fourre-tout : 8 méthodes à écrire, dont 7 « throw »
```

A · Le ratio

membres **utilisés** / membres **déclarés**, calculé **par client**. Sous 50 %, deux rôles sont mélangés.

B · Le faux minimal

Écris l'implémentation la plus bête possible. Si tu dois écrire 8 méthodes dont 7 vides → l'interface est trop large.

C · Le lint

Une méthode déclarée non implémentée, un paramètre jamais lu, un as de contournement : trois signaux gratuits.

RATIO DU CLIENT	LECTURE	GESTE
≥ 90 %	l'interface est calibrée pour ce client	ne rien faire
50 – 90 %	un rôle secondaire se cache	surveiller, mesurer sur les autres clients
< 50 %	**violation** : deux rôles mélangés	découper, ou retirer les méthodes mortes
0 % (aucune coche)	interface inutilisée	**la supprimer**

Le test, en dix lignes

```
// isp.test.ts — le faux ne doit décrire QUE le besoin du client
class FauxRepository implements Reader<Client> {      // 2 méthodes, pas 8
  findById(id: string) { return null }
  findPage(n: number, size: number) { return [] }
}
// si ce fichier a besoin d'un « throw new Error("not used") »,
// c'est l'interface qui est fausse — pas le test.
```

CE QUE TU MESURES	OÙ	SEUIL D'ALERTE
le ratio par client	revue de code, matrice	< 50 % → deux rôles mélangés
le nombre de méthodes du faux de test	les tests	> 3 → l'interface est trop large
le poids des imports d'un module	`import-cost`, bundle analyzer	> 100 Ko pour une fonction → interface locale
les membres déclarés non utilisés	ESLint `no-unused-vars`, TypeScript	> 0 → à supprimer

4 · LE DIAGNOSTIC

Anatomie de la violation

Une interface trop large produit toujours les mêmes trois symptômes.

Une interface, quatre implémentations qui la refusent

```
interface Gestionnaire { open · save · export · sync }
```

FichierLocal — pas d'export

```
export() { throw }
```

CloudDrive — synchro annulée

```
sync() { /* vide */ }
```

ArchiveZip — lecture seule

```
save() { throw }
```

ScannerRéseau — pas de stockage

```
save() { /* rien */ }
```

quatre implémentations, quatre méthodes refusées

Ce que le client subit

```
// il doit fournir 4 méthodes
// dont 3 qu'il n'appelle jamais
const faux = {
open: () => null,
save: () => {},
export: () => { throw 0 },
sync: () => {},
}
// 8 lignes de bruit par test
```

✗ **le test décrit l'interface**

✗ **pas le besoin du client**

Le symptôme :

des méthodes vides, des throw « not supported », et des faux de test de 40 lignes.

A · Signal n°1

Une méthode imposée que l'implémentation **refuse** : `throw`, corps vide, `return null`.

B · Signal n°2

Un **faux de test géant** : mocker coûte 15 méthodes. Personne ne le fait, donc personne ne teste.

C · Signal n°3

Le client **importe** tout un module pour une fonction, ou caste avec `as` pour contourner le type.

CE QUE LE CLIENT ÉCRIT	INTERFACE HONNÊTE	INTERFACE FOURRE-TOUT
le faux de test	2 méthodes	8 méthodes dont 6 `throw`
l'import	un contrat de 10 lignes	un module de 2 000 lignes
l'ajout d'une méthode	0 implémentation cassée	12 classes à mettre à jour

Détecter une violation en 30 secondes

1 `grep -rn "not supported" src/`

une méthode imposée et refusée

→ on regarde ce qu'il trouve

2 `grep -c "throw" fake-*.ts`

le faux de test trop gros

→ on regarde ce qu'il trouve

3 `eslint --rule no-unused-vars`

un paramètre jamais lu

→ on regarde ce qu'il trouve

CE QUE COÛTE LA VIOLATION	SANS ISP	AVEC ISP
écrire un test du client	8 stubs à écrire, dont 6 `throw`	**2 méthodes** utiles
ajouter une méthode à l'interface	12 classes à mettre à jour	**0** : on ajoute un rôle
comprendre ce que fait le client	8 méthodes à lire pour en trouver 2	le contrat **est** la réponse
brancher une implémentation partielle	impossible sans tout réécrire	capacité par capacité

Le vrai coût : personne n'écrit les huit stubs. Donc ce client n'est **pas testé** — et c'est exactement comme ça qu'une violation d'ISP se transforme en bug de production six mois plus tard.

5 · LA CIBLE

Anatomie du respect

Chaque client reçoit exactement le contrat qu'il utilise — et les implémentations composent les rôles.

Après : un rôle par besoin, et la composition pour les combiner

interface Opener `open()`		**interface Saver** `save()`		**interface Exporter** `export()`		
FichierLocal	implements	**Opener, Saver**	✓ open	✓ save		
ArchiveZip	implements	**Opener**	✓ open			
CloudDrive	implements	**Opener, Saver, Exporter**	✓ open	✓ save	✓ export	

0 méthode refusée · chaque client dépend d'une seule interface

Les 3 gestes de correction, dans cet ordre

1 Séparer

d'une interface fourre-tout, extraire les rôles réellement utilisés par chaque client.

2 Nommer par le besoin

Chargeable plutôt que PaymentProvider : le nom dit à quoi ça sert.

3 Composer

qui a besoin de deux rôles les combine ; qui doit tout offrir implémente plusieurs.

> **Le critère de réussite :** dans le fichier du client, **aucune méthode** de l'interface n'est laissée de côté, et le faux de test tient en **deux lignes**.

La composition, quand un client a besoin de plusieurs rôles

```
interface Reader<T> { findById(id: string): T | null; findPage(n: number, s: number): T[] }
interface Writer<T> { save(e: T): void; delete(id: string): void }

// un besoin qui couvre les deux : on compose, on ne gonfle pas
interface Full<T> extends Reader<T>, Writer<T> {}

// et côté implémentation :
class ClientRepo implements Full<Client> { … }

// côté client, on ne demande JAMAIS Full si l'on ne lit que :
function liste(r: Reader<Client>) { return r.findPage(0, 20) }
```

Où déclarer l'interface ? Chez le client, pas chez l'objet

```
// liste.controller.ts — le contrat est défini PAR le client
interface ClientReader { findPage(n: number, size: number): Client[] }

class ListeController {
  constructor(private repo: ClientReader) {}   // 1 méthode
}

// client-repo.ts — l'implémentation s'y conforme, elle ne le dicte pas
class ClientRepo implements ClientReader { … }
// c'est la « role interface » : elle vit à côté de celui qui l'utilise
```

A · Le client maigrit

Moins de dépendances = moins de mocks, moins d'imports, moins de recompilations.

B · L'implémentation reste libre

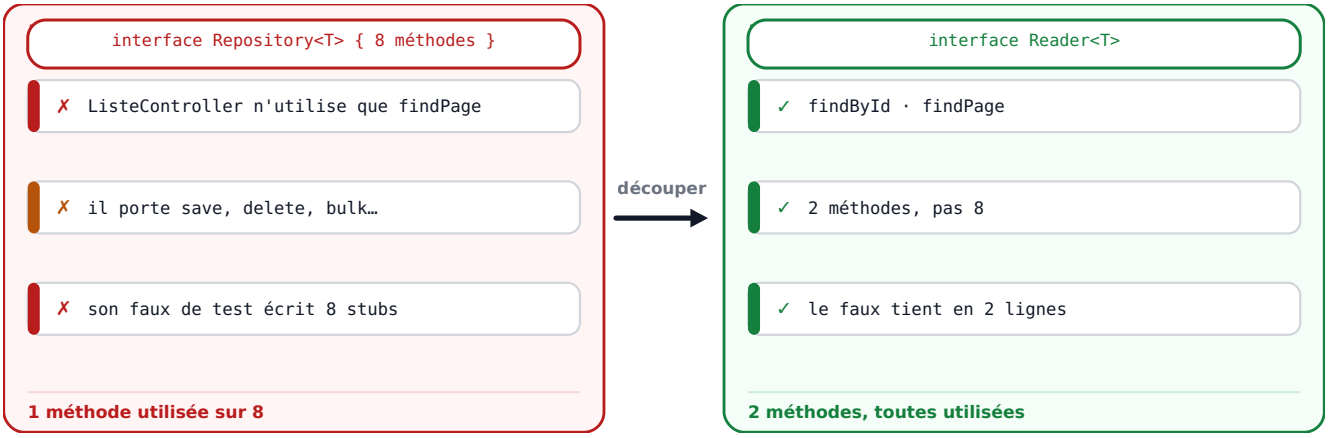
Une classe peut implémenter dix rôles. C'est côté client que la vue se rétrécit.

C · Les tests deviennent possibles

Un faux de deux lignes, c'est un test qu'on écrit vraiment.

1 Le Repository fourre-tout

1 client, 8 méthodes déclarées



AVANT

```
// repository.ts
interface Repository<T> {
  findById(id: string): T | null
  findAll(): T[]
  findPage(n: number, size: number): T[]
  save(e: T): void
  update(e: T): void
  delete(id: string): void
  count(): number
  bulkInsert(list: T[]): void
}

class ListeController {
  constructor(private repo: Repository<Client>) {}
  page(n: number) { return this.repo.findPage(n, 20) }
}
```

APRÈS

```
// reader.ts — le rôle de ce client
interface Reader<T> {
  findById(id: string): T | null
  findPage(n: number, size: number): T[]
}

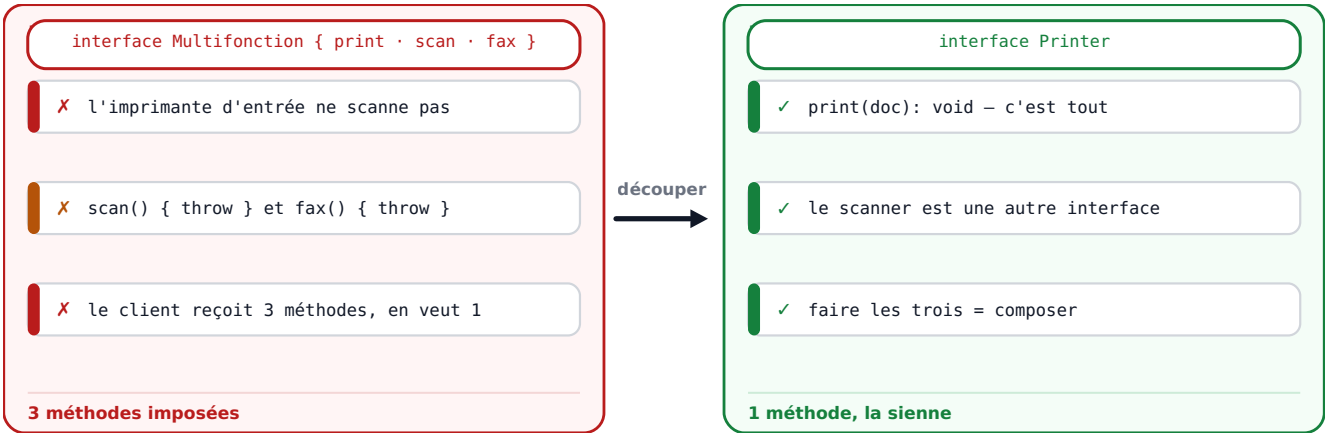
class ListeController {
  constructor(private repo: Reader<Client>) {}
  page(n: number) { return this.repo.findPage(n, 20) }
}

// ClientRepo implémente Reader, Writer, Pager
// le contrôleur n'en voit que deux méthodes
```

À retenir : le contrôleur ne peut plus appeler `save()`, non par discipline mais **parce que le type le lui interdit**. Et son faux de test passe de 8 méthodes à 2.

2 L'imprimante qui scanne et faxe

les capacités qui ne vont pas ensemble



AVANT

```
// imprimante.ts
interface Multifonction {
  print(d: Document): void
  scan(): Image
  fax(d: Document, num: string): void
}

class ImprimanteEntree implements Multifonction {
  print(d: Document) { /* ok */ }
  scan(): Image { throw new Error("pas de scanner") }
  fax() { throw new Error("pas de fax") }
}
```

APRÈS

```
// imprimante.ts
interface Printer { print(d: Document): void }
interface Scanner { scan(): Image }
interface Fax     { fax(d: Document, num: string): void }

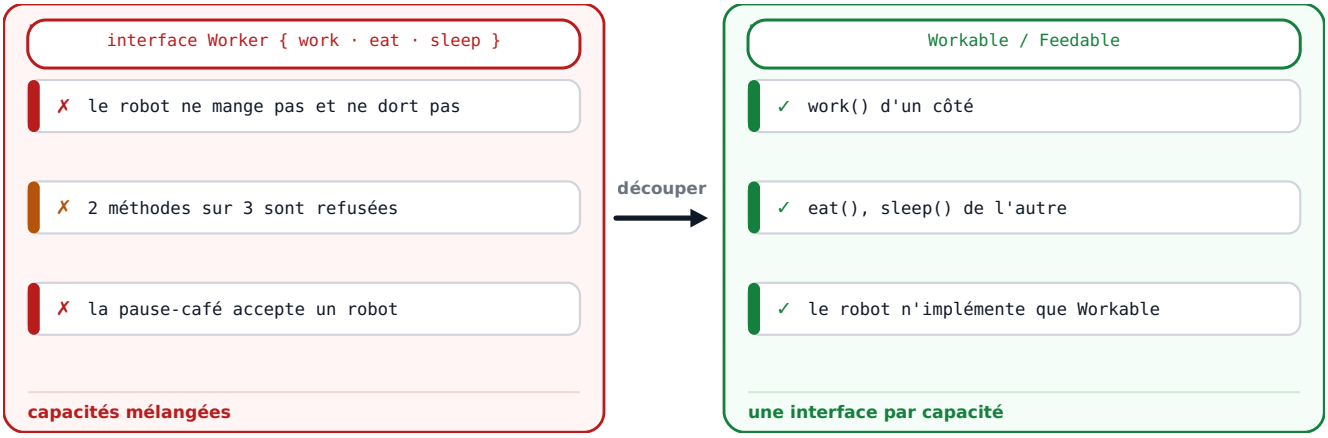
class ImprimanteEntree implements Printer { … }
class MultifonctionPro  implements Printer, Scanner, Fax { … }

// celui qui imprime : function imprimer(p: Printer)
```

À retenir : c'est l'exemple d'école, et il est vrai depuis 1995 : chaque capacité absente devient un `throw` dans une classe qui n'a rien demandé.

3 L'humain et le robot

le sous-type qui n'a pas les mêmes besoins



AVANT

```
// worker.ts
interface Worker {
  work(): void
  eat(): void
  sleep(hours: number): void
}

class Robot implements Worker {
  work() { /* ok */ }
  eat() { throw new Error("je ne mange pas") }
  sleep() { throw new Error("je ne dors pas") }
}
```

APRÈS

```
// worker.ts
interface Workable { work(): void }
interface Feedable { eat(): void }
interface Restable { sleep(hours: number): void }

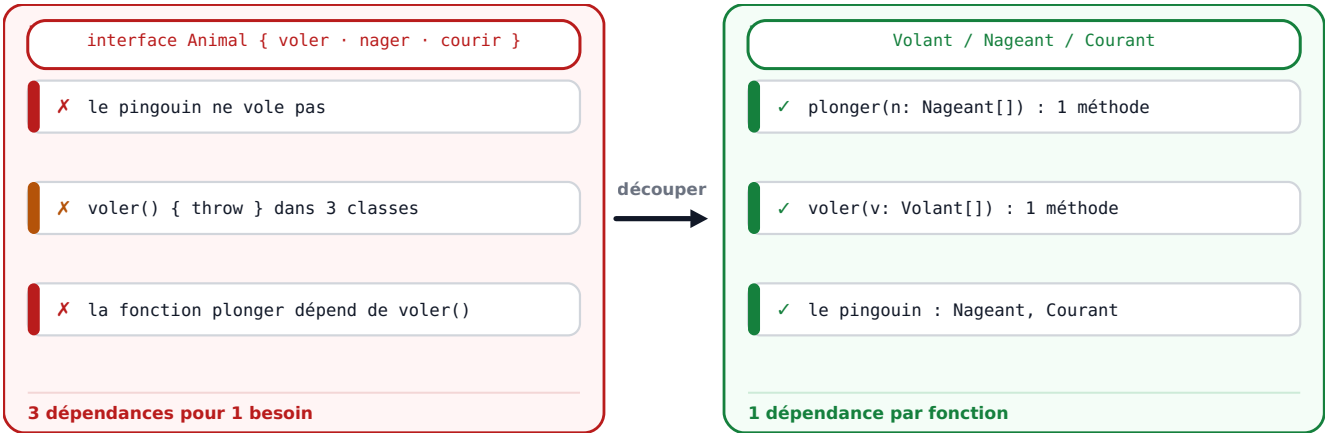
class Humain implements Workable, Feedable, Restable { … }
class Robot  implements Workable { … }

// function pauseCafe(agents: Feedable[]) { … }
// un Robot ne peut plus y entrer
```

À retenir : on découpe par **capacité**, pas par « ce qu'un travailleur est censé faire ». Bénéfice secondaire : la pause-café ne peut plus recevoir un robot.

4 Voler, nager, courir

la fonction qui dépend de tout l'objet



AVANT

```
// animal.ts
interface Animal {
  voler(): void
  nager(): void
  courir(): void
}

class Pingouin implements Animal {
  voler() { throw new Error("non") }
  nager() { /* ok */ }
  courir() { /* ok */ }
}
// function plonger(a: Animal[]) { a.forEach(x => x.nager()) }
```

APRÈS

```
// animal.ts
interface Volant  { voler(): void }
interface Nageant { nager(): void }
interface Courant { courir(): void }

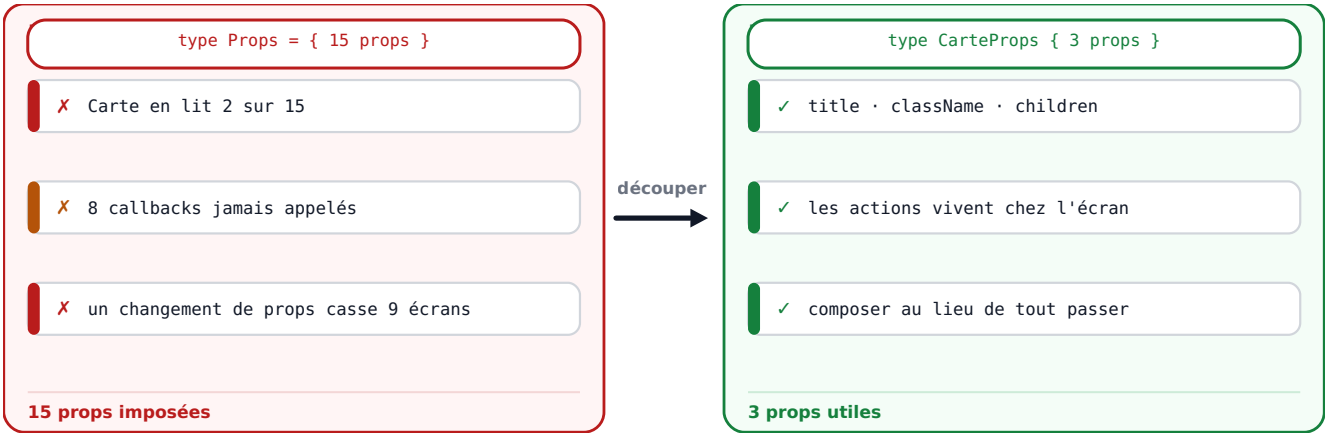
class Pingouin implements Nageant, Courant { … }
class Aigle    implements Volant, Courant { … }

// function plonger(n: Nageant[]) { … }  → 1 besoin
// function envoler(v: Volant[])  { … }  → 1 besoin
```

À retenir : ici ISP et LSP se croisent — mais la question d'ISP est **côté client** : « ma fonction `plonger` a-t-elle besoin de connaître `voler()` ? » Non, donc elle ne le déclare pas.

5 Les props d'un composant

ISP ne parle pas que de méthodes



AVANT

```
// carte.tsx
type Props = {
  title: string; subtitle?: string; image?: string
  onSave?: () => void; onDelete?: () => void
  onShare?: () => void; isAdmin?: boolean
  theme?: "light" | "dark"; locale?: string
  loading?: boolean; error?: string; retry?: () => void
  className?: string; style?: CSSProperties; testId?: string
}
function Carte(p: Props) {
  return <div className={p.className}>{p.title}</div>
}
```

APRÈS

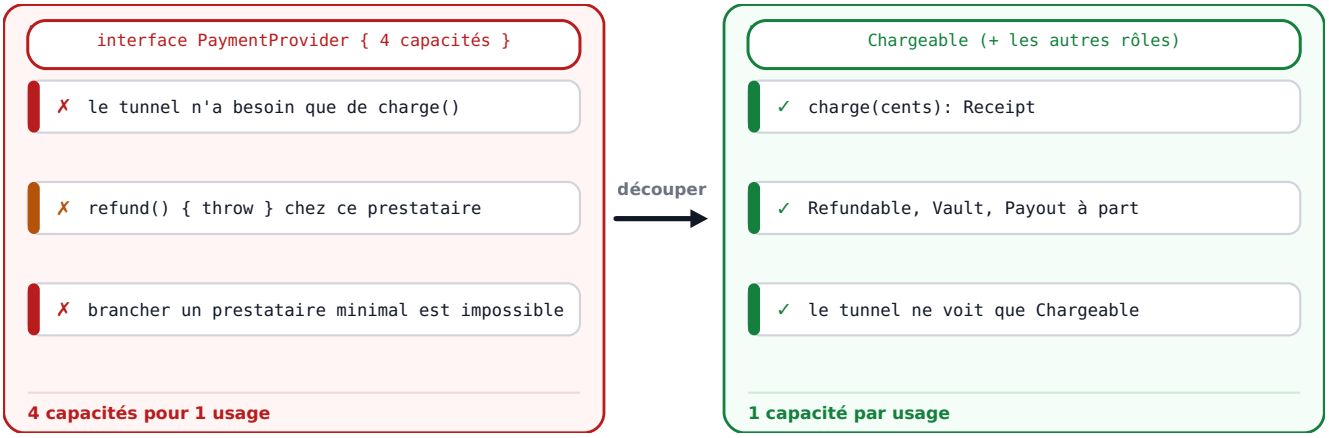
```
// carte.tsx
type CarteProps = {
  title: string
  className?: string
  children?: ReactNode
}
function Carte({ title, className, children }: CarteProps) {
  return <div className={className}>{title}{children}</div>
}

// les actions vivent là où elles servent :
<Carte title={t}><BoutonSupprimer id={id} /></Carte>
```

À retenir : ISP s'applique aussi aux **données**. Un composant qui reçoit 15 props en utilise 2 : les 13 autres sont une dépendance imposée — et un re-rendu de plus.

6 Le prestataire de paiement

ce que le tunnel utilise vraiment



AVANT

```
// paiement.ts
interface PaymentProvider {
  charge(cents: number): Receipt
  refund(id: string): void
  saveCard(card: Card): Token
  payout(iban: string, cents: number): void
}

// le tunnel de commande n'appelle QUE charge()
function payer(p: PaymentProvider, cents: number) {
  return p.charge(cents)
}
```

APRÈS

```
// paiement.ts
interface Chargeable { charge(cents: number): Receipt }
interface Refundable { refund(id: string): void }
interface Vault      { saveCard(card: Card): Token }

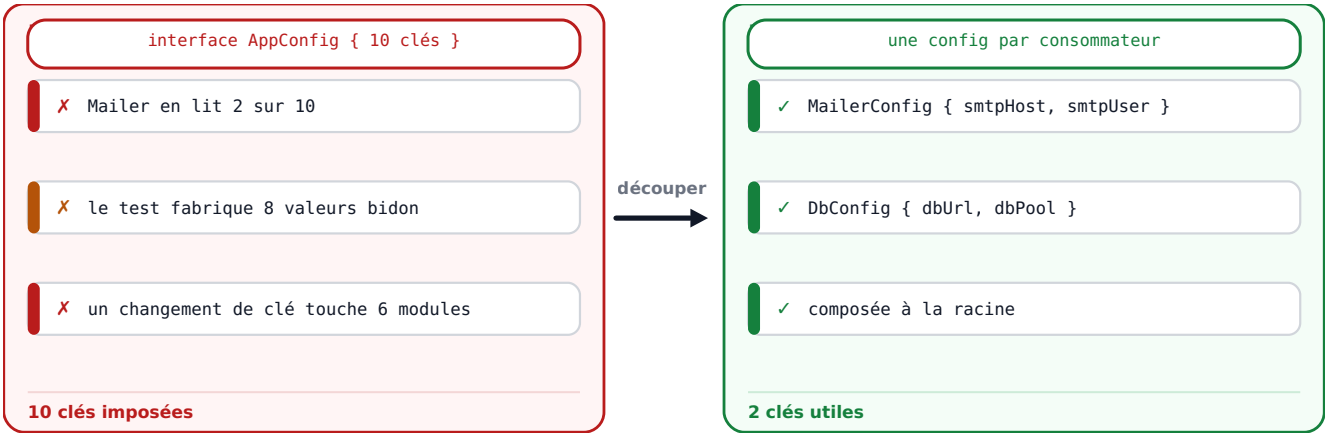
function payer(p: Chargeable, cents: number) {
  return p.charge(cents)
}

// StripeProvider : Chargeable, Vault, Refundable
// PetitPrestataire : Chargeable — suffisant pour le tunnel
```

À retenir : une interface nommée d'après le **fournisseur** (`PaymentProvider`) décrit tout ce qu'il sait faire ; une interface nommée d'après le **besoin** (`Chargeable`) décrit ce dont le client a besoin.

7 La configuration monolithique

les dépendances de données aussi



AVANT

```
// config.ts
interface AppConfig {
  dbUrl: string; dbPool: number
  smtpHost: string; smtpUser: string
  stripeKey: string; s3Bucket: string
  jwtSecret: string; logLevel: string
  featureFlags: Record<string, boolean>
  cacheTtl: number
}

class Mailer {
  constructor(private cfg: AppConfig) {}   // il veut 2 clés
}
```

APRÈS

```
// config.ts — chaque module déclare son bout
interface MailerConfig { smtpHost: string; smtpUser: string }
interface DbConfig     { dbUrl: string; dbPool: number }

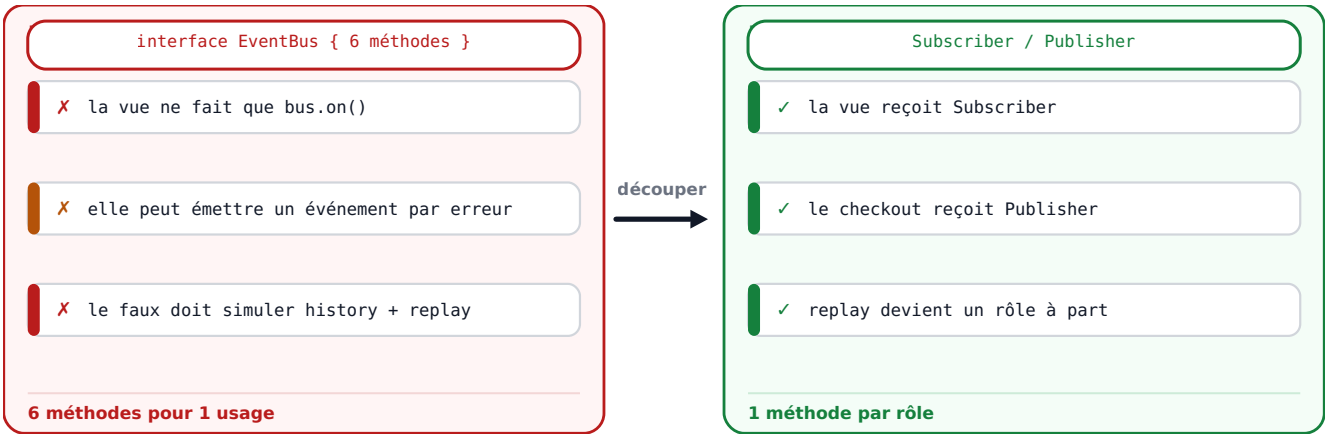
class Mailer {
  constructor(private cfg: MailerConfig) {}
}

// composition à la racine :
// new Mailer({ smtpHost: env.HOST, smtpUser: env.USER })
```

À retenir : c'est le gain le plus **immédiat** et le plus sous-estimé : un test de `Mailer` n'a plus à inventer dix valeurs pour en utiliser deux.

8 Le bus d'événements

lire et écrire ne sont pas le même rôle



AVANT

```
// bus.ts
interface EventBus {
  on(e: string, h: Handler): void
  off(e: string, h: Handler): void
  once(e: string, h: Handler): void
  emit(e: string, p: unknown): void
  history(): Event[]
  replay(from: number): void
}

class VuePanier {
  constructor(private bus: EventBus) {}   // .on() seulement
}
```

APRÈS

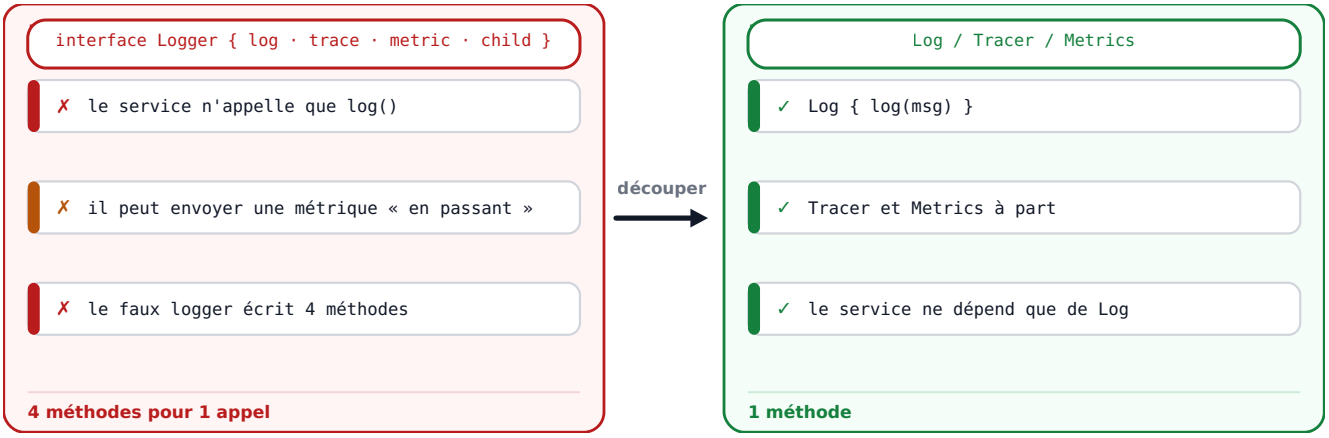
```
// bus.ts
interface Subscriber { on(e: string, h: Handler): void }
interface Publisher  { emit(e: string, p: unknown): void }
interface Replayable { history(): Event[]; replay(from: number): void }

class VuePanier {
  constructor(private bus: Subscriber) {}
}
class Checkout {
  constructor(private bus: Publisher) {}
}
// la vue ne peut plus émettre : c'est le type qui l'empêche
```

À retenir : une vue ne peut plus **émettre** un événement par accident : elle ne reçoit que la moitié du bus dont elle a besoin. Le faux de test fait une méthode.

9 Le logger qui trace et métrique

une interface, trois responsabilités



AVANT

```ts
// logger.ts
interface Logger {
  log(m: string): void
  debug(m: string): void
  trace(): Span
  metric(name: string, v: number): void
  child(ctx: object): Logger
}

class ServiceMetier {
  constructor(private log: Logger) {}   // log() seulement
}
```

APRÈS

```ts
// logger.ts
interface Log     { log(m: string): void }
interface Tracer  { trace(): Span }
interface Metrics { metric(name: string, v: number): void }

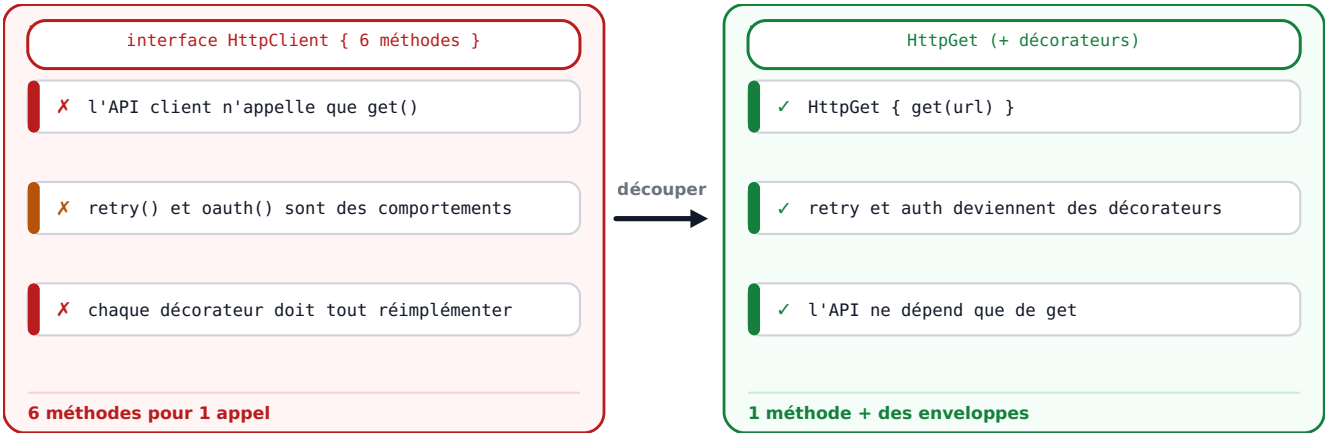
class ServiceMetier {
  constructor(private log: Log) {}
}

// ImplLogger implémente les trois
// le test : class FauxLog { lignes: string[] = [] }
```

À retenir : en test unitaire, le faux logger fait **une** ligne. Et le service métier ne peut plus envoyer une métrique « au passage » — c'est une responsabilité, pas un effet de bord.

10 Le client HTTP couteau-suisse

les capacités transverses dans l'interface



AVANT

```ts
// http.ts
interface HttpClient {
  get(u: string): Promise<Response>
  post(u: string, b: unknown): Promise<Response>
  upload(u: string, f: File): Promise<void>
  stream(u: string): AsyncIterable<Chunk>
  oauth(token: string): void
  retry(n: number): void
}

class ApiClients {
  constructor(private http: HttpClient) {}   // get() seulement
}
```

APRÈS

```ts
// http.ts
interface HttpGet   { get(u: string): Promise<Response> }
interface HttpWrite {
  post(u: string, b: unknown): Promise<Response>
}

class AvecRetry implements HttpGet { … }   // décorateur
class AvecAuth  implements HttpGet { … }   // décorateur

class ApiClients {
  constructor(private http: HttpGet) {}
}
// new ApiClients(new AvecAuth(new AvecRetry(impl)))
```

À retenir : `retry` et `oauth` ne sont pas des **méthodes**, ce sont des **comportements enveloppants**. Les sortir de l'interface libère tous les clients qui n'en veulent pas.

6 · TOUR D'HORIZON

Le reste de la boîte, en un coup d'œil

Neuf interfaces fourre-tout que tu as forcément croisées : à gauche le catalogue, à droite les rôles.

UserService
15 méthodes, 5 cas d'usage
	`register · login`
	`reset · changeRole`
	`export · …`
		`RegisterUser`
		`FindUser`
		`ChangeRole`
			un service = 5 rôles
			une interface par cas d'usage

Vehicle
drive / fly / sail
	`interface Vehicle`
	`drive()`
	`fly() · sail()`
		`Drivable`
		`Flyable`
		`Sailable`
			3 capacités
			3 interfaces

IDocument
lire, écrire, signer
	`read · write`
	`print · sign`
	`(4 en 1)`
		`Readable`
		`Writable`
		`Signable`
			un viewer écrit ?
			plus maintenant

Storage
upload, delete, share
	`upload → download`
	`delete → share`
	`list`
		`ReadStore`
		`WriteStore`
		`ShareStore`
			un CDN en lecture
			devient branchable

Metrics
counter/gauge/histogram
	`counter() · gauge()`
	`histogram()`
	`flush()`
		`Counter`
		`Gauge`
		`Flusher`
			les faux : 1 méthode
			au lieu de 4

IdGenerator
next / peek / reset
	`next() · peek()`
	`reset() · seed()`
	`(4 en 1)`
		`NextId`
		`Resettable`
			le générateur
			n'expose que next()

Notifier
email / sms / push
	`send · broadcast`
	`subscribe`
	`templates · track`
		`Sender`
		`Subscribable`
			un client veut
			un seul canal

FileSystem
read / write / watch
	`read · write`
	`watch · copy`
	`chmod`
		`ReadFs`
		`WriteFs`
		`Watcher`
			un lecteur ne peut
			plus écrire

Auth
login, refresh, roles
	`login · logout`
	`refresh · roles()`
	`permissions()`
		`Authenticator`
		`SessionRefresher`
		`Authorizer`
			l'UI rafraîchit ?
			plus maintenant

A · Le réflexe

Un `throw` dans une implémentation est un aveu : cette méthode n'a rien à faire dans l'interface.

B · La question

« Ce client utilise-t-il **tout** ce que l'interface lui impose ? » Si non, l'interface n'est pas la sienne.

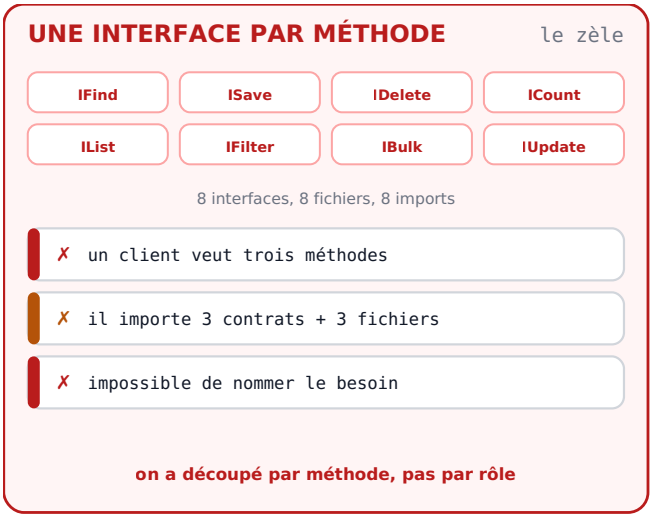
C · Le gain caché

Moins de dépendances = tests plus simples, builds plus rapides, et des faux de trois lignes qu'on écrit vraiment.

Le fil rouge de ces neuf cas : l'interface n'est jamais découpée « pour faire propre » — elle est découpée **parce qu'un client précis** n'avait besoin que d'une partie. Sans ce client, pas de découpe.

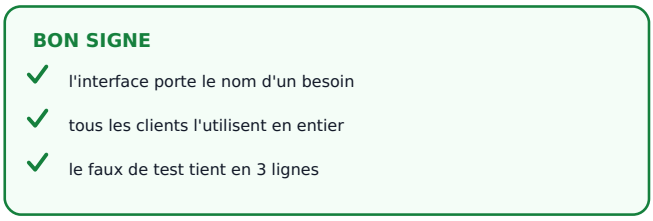
L'explosion d'interfaces

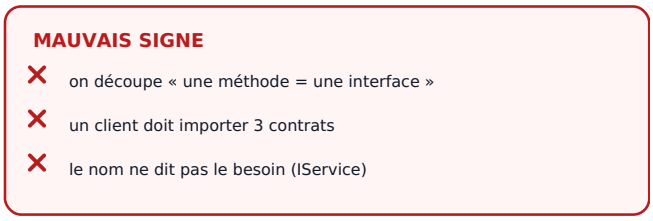
Le zèle inverse : découper « une méthode = une interface » rend le code illisible. ISP dit **par rôle client**, pas **par méthode**.





Le bon grain : ni une interface fourre-tout, ni une interface par méthode





Trop fin — le zèle

```
interface IFind   { find(id): T }
interface ISave   { save(e: T): void }
interface ICount  { count(): number }
// … 8 fichiers, 8 imports
// aucun ne dit « Reader »
// un client en importe trois
```

Juste — un rôle par besoin

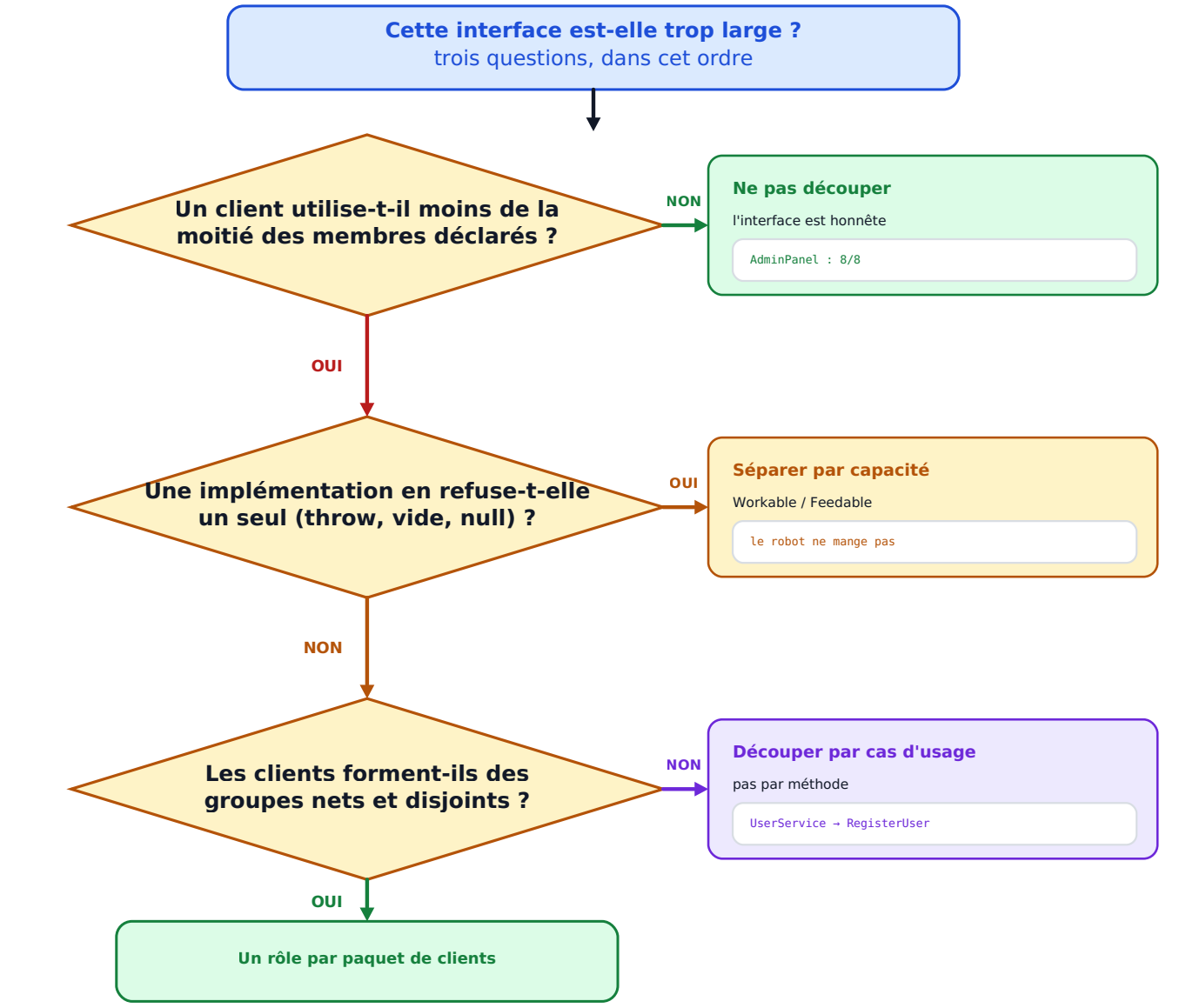
```
interface Reader<T> {
  find(id: string): T | null
  findPage(n: number, s: number): T[]
}
// … 3 interfaces nommées par le besoin
// un client importe celle qu'il utilise
// et compose s'il en faut deux
```

La différence en une phrase : `IFind` décrit **une méthode** ; `Reader` décrit **un besoin**. Le premier oblige le client à assembler les morceaux, le second lui donne exactement ce qu'il utilise.

LE PIÈGE	CE QU'IL RÉVÈLE	LE GESTE
une interface par méthode (`IFind`, `ISave`…)	on a découpé par implémentation	regrouper par **rôle client**
confondre ISP et SRP	SRP parle de **qui change**, ISP de **qui dépend**	deux questions distinctes
ne découper que les méthodes	les **données** aussi sont des dépendances	découper les DTO et les props
une interface vide (`Marker`)	aucune garantie, juste une étiquette	un champ discriminant
un client qui a besoin de 6 rôles	c'est **le client** qui est trop gros	là, c'est SRP qui parle
une interface gardée « pour la compatibilité »	elle grossit à chaque nouveau besoin	versionner : v2 découpée, v1 dépréciée

Faut-il découper cette interface ?

Trois questions, dans cet ordre. La première évite 90 % des découpages inutiles.



Dans le doute : écris le faux de test du client.

S'il dépasse trois méthodes, l'interface est trop large.

Repository → Reader
le contrôleur ne lit que

Worker → Workable
le robot ne mange pas

Props → 3 props
la carte n'en utilise que 2

A · Le faux positif n°1

« Les interfaces doivent être petites » n'est pas le critère : **l'usage du client** l'est.

B · Le faux positif n°2

Un client qui a besoin de **six** rôles : ce n'est pas l'interface qui est trop large, c'est lui qui fait trop.

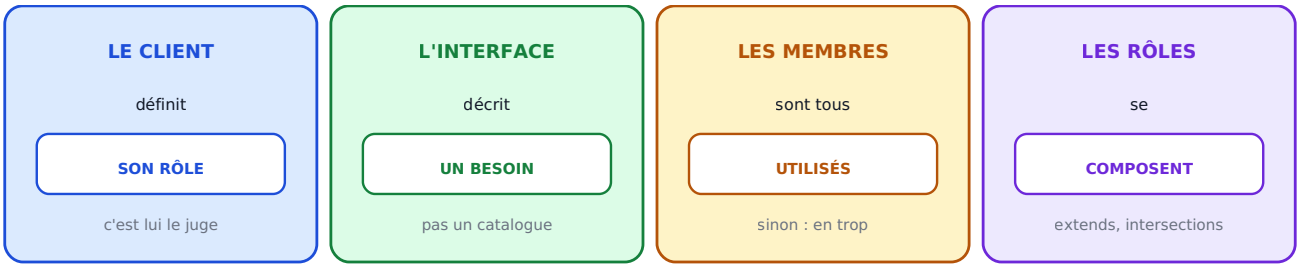
C · Le faux positif n°3

Découper **avant** d'avoir deux clients : on invente des rôles que personne n'utilise.

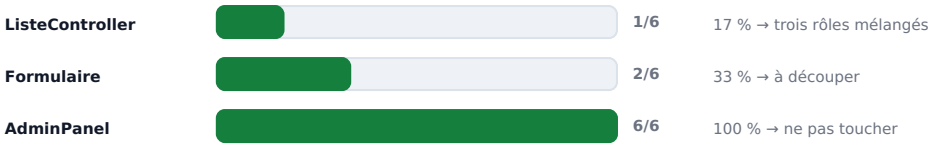
Le raccourci : ISP ne demande pas « cette interface est-elle petite ? » mais « **ce client** dépend-il de choses qu'il n'utilise pas ? ». La réponse se mesure sur la matrice, client par client.

Le principe en une image, et le chiffre qui le résume

Si tu retiens une page, c'est celle-ci.



Le réflexe chiffré : le ratio du client



En une phrase ?

Un client ne doit dépendre que de ce qu'il utilise.

Qui est le juge ?

Le client : c'est son besoin qui définit l'interface, pas les capacités de l'objet.

Les 4 règles ?

Par rôle · minimale · par capacité · par composition.

Les signaux ?

`throw "not supported"` · un faux de test géant · un as pour contourner un type · un import de 2 000 lignes pour une fonction.

Les 3 gestes ?

Séparer → nommer par le besoin → composer.

ISP vs SRP ?

SRP regarde qui change ; ISP regarde qui dépend.

ISP vs LSP ?

LSP : tiens la promesse. ISP : ne promets pas plus que ce dont le client a besoin.

ISP vs DIP ?

DIP : dépends d'abstractions. ISP : de **petites** abstractions, taillées pour ton client.

Le piège n°1 ?

Croire qu'ISP = « une méthode par interface ». C'est le zèle inverse (page 13).

CHECKLIST AVANT DE VALIDER UNE INTERFACE	SI NON →
Chaque client l'utilise-t-il **en entier** ?	extraire le rôle de ce client
Aucune implémentation ne refuse un membre ?	séparer par capacité
Le nom dit-il le **besoin** (Reader, Chargeable) ?	renommer avant de découper
Le faux de test tient-il en 3 méthodes ?	l'interface est trop large
Une méthode n'est-elle utilisée par **personne** ?	la supprimer

Les quatre principes côte à côte



Et si tu oublies tout le reste : avant d'ajouter une méthode à une interface, demande-toi **qui** va l'appeler. Si la réponse est « personne pour l'instant », elle n'a rien à y faire.

10 · SITUER

ISP dans SOLID

Les cinq principes ne se récitent pas : ils se répondent. ISP est celui qui fixe la **taille** des abstractions.

ISP ne vit pas seul : trois principes le bordent



SRP → ISP	SRP découpe les responsabilités ; ISP découpe les dépendances. Les deux répondent à « qui utilise quoi ».
LSP → ISP	LSP exige qu'une promesse soit tenue ; ISP évite d'en promettre plus que nécessaire. Moins de promesses = moins de risque.
DIP → ISP	DIP dit « dépends d'abstractions » ; ISP ajoute : « petites, et taillées pour ce client ».

À retenir : ISP est le principe qui rend les trois autres tenables — c'est lui qui fixe la taille des abstractions.

A · Sans ISP

Un `PrincipeOuvertFerme` sur une interface fourre-tout oblige à modifier les implémentations qui n'ont rien demandé — OCP devient inapplicable.

B · Sans ISP, pas de LSP

Plus une interface promet, plus un sous-type risque de ne pas tenir. ISP réduit la surface de promesse, donc le risque de violation.

C · Avec ISP, DIP tient

Dépendre d'une abstraction de 40 méthodes n'inverse rien du tout. ISP est la condition pratique de l'inversion de dépendance.

PRINCIPE	LA QUESTION À LAQUELLE IL RÉPOND	CE QU'IL DÉCOUPE
SRP	« quelles responsabilités cette classe porte-t-elle ? »	les **raisons de changer**
OCP	« puis-je ajouter un cas sans modifier l'existant ? »	les **points d'extension**
LSP	« puis-je remplacer sans casser ? »	les **contrats et leurs promesses**
ISP	« de quoi ce client a-t-il **vraiment** besoin ? »	les **dépendances**
DIP	« de quoi dépend-on : d'un détail ou d'une abstraction ? »	les **directions de dépendance**

L'ordre de lecture : SRP découpe le **code**, ISP découpe ce que le **client voit** de ce code. On peut avoir des classes parfaitement SRP et une interface fourre-tout — c'est même le cas le plus fréquent.

PRINCIPE	CE QU'IL NE DIT PAS	LE CONTRESENS COURANT
SRP	qu'une classe doit être petite	« une méthode par classe »
OCP	qu'il faut une abstraction pour tout	« une interface pour chaque classe »
LSP	qu'il faut hériter	« un sous-type = `extends` »
ISP	que les interfaces doivent être petites	« une interface par méthode » (page 13)
DIP	qu'il faut un conteneur d'injection	« injecter = DIP »

SRP ou ISP ?

SRP : cette classe a-t-elle une seule raison de changer ? ISP : ce client dépend-il de choses inutiles ?

Et si tous les clients utilisent tout ?

Alors l'interface est honnête : ne pas la découper. Une grande interface peut être un bon contrat.

ISP pour les données ?

Oui : un DTO de 30 champs dont le client lit 2 est une dépendance imposée, exactement comme une méthode.

Six refactorings, du plus simple au plus profond

ISP ne se « pense » pas, il se **refactore** : commence toujours par supprimer, puis découpe.

Les 6 refactorings d'ISP, du plus simple au plus profond



LE REFACTORING	QUAND	RISQUE SI ON SAUTE L'ÉTAPE
supprimer le membre mort	0 appel dans tout le code	on découpe autour de choses inutiles
extraire le rôle	un client n'utilise qu'une partie	le client garde la dépendance entière
déplacer vers le client	le contrat est propre au client	le contrat reste dicté par l'objet
séparer par capacité	une implémentation refuse	les `throw` restent
composer	un client a besoin de deux rôles	on regonfle une interface
envelopper	le besoin est transverse (retry, auth)	chaque client réimplémente

Avant — l'interface est écrite du point de vue de l'objet

```
interface Repository<T> {
  findById(id: string): T | null
  findPage(n: number, s: number): T[]
  save(e: T): void
  delete(id: string): void
}
// le contrôleur importe les 4
class ListeController {
  constructor(private repo: Repository<Client>) {}
}
```

Après — le rôle est extrait, puis déplacé chez le client

```
// liste.controller.ts
interface ClientReader {
  findPage(n: number, s: number): Client[]
}

class ListeController {
  constructor(private repo: ClientReader) {}
}

// client-repo.ts : implements ClientReader, Writer
// l'objet ne dicte plus le contrat, il s'y conforme
```

Par où commencer ?

Par **supprimer** : la moitié des violations disparaissent quand on enlève les membres que personne n'appelle.

Combien de rôles ?

Autant que de **paquets de colonnes** dans la matrice — ni un par client, ni un par méthode.

Quand s'arrêter ?

Quand chaque client utilise **tout** ce que son interface déclare. Là, on a fini.

L'ordre compte. Découper une interface qui contient encore des méthodes mortes, c'est ranger une pièce avant de l'avoir vidée : on découpe autour du bruit, et on finit avec six interfaces dont trois ne servent à personne.

Un refactoring d'ISP réussi ne s'annonce pas. Aucun appelant ne change, aucun test ne casse : seule la liste des imports du client rétrécit. Si tu dois modifier le code client, tu n'as pas découpé l'interface — tu l'as renommée.

12 · S'ENTRAÎNER

Cinq extraits, cinq gestes

Pour chaque extrait : quel client, combien de membres utilisés, quel geste ?

À toi : cinq extraits, cinq gestes

1

```
interface Forme {
aire()
perimetre()
dessiner(ctx)
}
```

symptôme
Le client ne calcule que aire()

geste
extraire Calculable

2

```
class FauxRepo
implements Repo {
find() { null }
save() { throw }
… 6 de plus
```

symptôme
Le faux est énorme

geste
réduire l'interface du client

3

```
interface Cache {
get(k): T
set(k, v): void
clear(): void
stats(): Stats
}
```

symptôme
Le lecteur n'appelle que get()

geste
interface CacheReader

4

```
type Props = {
onSave, onDelete,
onShare, isAdmin,
theme, locale
}
```

symptôme
Le composant en lit 2

geste
3 props + composition

5

```
interface Sender {
mail(m)
sms(m)
push(m)
slack(m)
```

symptôme
Un client n'envoie que mail

geste
interface Mailable

Solutions en bas de page — cherche d'abord, c'est là que ça rentre.

Vrai ou faux ?

ISP demande que les interfaces soient courtes.	**FAUX**	— le critère est l'usage du client
Une interface utilisée en entier par tous les clients est correcte.	**VRAI**	— taille ≠ critère
Un client peut dépendre de 3 rôles s'il en a besoin.	**VRAI**	— c'est alors SRP qui parle
Les props d'un composant sont hors du champ d'ISP.	**FAUX**	— les données aussi sont des dépendances
Découper une méthode par interface est la bonne pratique.	**FAUX**	— c'est le zèle (page 13)
Un faux de test de 12 méthodes est un signal ISP.	**VRAI**	— le symptôme le plus fiable

Solutions : 1 → extraire Calculable ({ aire() }) · 2 → l'interface du test devient le contrat du client (2 méthodes) · 3 → CacheReader pour le lecteur, CacheAdmin pour le reste · 4 → 3 props + `children`, les actions deviennent des composants · 5 → Mailable (1 méthode) et Sender pour qui veut tout.

Corrigé n°1 — extraire Calculable

```
interface Calculable { aire(): number }   // 1 membre

function total(f: Calculable[]) {         // le client
  return f.reduce((s, x) => s + x.aire(), 0)
}
// Forme garde aire + perimetre + dessiner
// pour ceux qui en ont besoin
```

Corrigé n°5 — un rôle par canal

```
interface Mailable { mail(m: Message): void }

class RelanceFacture {                    // 1 canal
  constructor(private sender: Mailable) {}
}

// Sender reste pour qui envoie sur 4 canaux
// interface Sender extends Mailable, Smssable, …
```

Exercice 6

Une interface `Storage` de 12 méthodes, utilisée par un CDN en lecture. Quel geste ? → extraire `ReadStore`.

Exercice 7

Un service qui a besoin de 6 rôles différents. Découpe-t-on l'interface ? → non : c'est le **service** qui est trop gros (SRP).

Exercice 8

Une interface où **chaque** client coche toutes les cases. Que faire ? → rien. C'est un contrat honnête.

AUTO-ÉVALUATION	OUI	NON
j'ai écrit la matrice clients × méthodes pour cette interface	→ on découpe ce qui reste	→ commence par là, 3 minutes
mon faux de test fait 2 ou 3 méthodes	→ c'est bon	→ l'interface est trop large
aucune implémentation ne lève « not supported »	→ c'est bon	→ séparer par capacité
chaque interface porte le nom d'un besoin	→ c'est bon	→ renommer avant de découper
aucun client n'importe trois contrats pour un usage	→ c'est bon	→ regrouper par rôle

13 · IMPRIMER

La fiche mémo

Une page, à coller à côté de l'écran : la règle, le chiffre, les signaux, l'arbre.

La fiche à imprimer : tout ISP sur une page

LA RÈGLE

Aucun client ne dépend de méthodes qu'il **n'utilise pas.**

→ l'interface appartient au client

LES 4 RÈGLES

- ✓ par rôle de client
- ✓ minimale
- ✓ par capacité
- ✓ par composition

LE CHIFFRE

≥ 90 %	calibrée
50-90 %	à surveiller
< 50 %	violation

membres utilisés / déclarés

Signaux → gestes

✗ `throw « not supported »`	→ ✓	**séparer la capacité**
✗ `override vide / rien du tout`	→ ✓	**composer ou supprimer**
✗ `faux de test de 12 méthodes`	→ ✓	**extraire le rôle du client**
✗ `un cast pour appeler une méthode`	→ ✓	**le client n'a pas son interface**
✗ `import de 2 000 lignes pour 1 fonction`	→ ✓	**interface locale à la frontière**
✗ `nom : Manager, Service, Helper, IEntity`	→ ✓	**nommer par le besoin**

L'arbre de décision (page 14), en trois lignes

1. Un client utilise-t-il moins de la moitié ?	**→ oui : découper**
2. Une implémentation refuse-t-elle un membre ?	**→ oui : séparer par capacité**
3. Les clients forment-ils des groupes disjoints ?	**→ oui : un rôle par groupe**

ISP en une phrase ?

Aucun client ne doit dépendre de méthodes qu'il n'utilise pas.

Le test ?

Écris le faux de test du client : s'il dépasse trois méthodes, l'interface est trop large.

Le geste ?

Supprimer → extraire le rôle → nommer par le besoin → composer.

Le squelette d'une interface ISP

```
// 1. nommer par le besoin du client
interface RelanceClient {
// 2. un verbe par usage réel
  trouverImpayes(): Facture[]
  envoyerRelance(f: Facture): void
}
// 3. ni plus, ni moins : si un client
//    n'utilise pas une ligne, elle sort
```

Les 4 pièges à ne pas confondre

```
// ✗ une méthode par interface (IFind, ISave)
// ✗ une interface vide (Marker)
// ✗ découper les méthodes mais pas les DTO
// ✗ découper avant d'avoir deux clients
// ↑ les quatre rendent le code plus dur,
//   pas plus simple : ISP vise le client
```

LES 10 GESTES EXPRESS, DANS L'ORDRE OÙ ON LES APPLIQUE	PAGE
supprimer les membres que personne n'appelle	13
écrire le faux de test du client — s'il dépasse 3 méthodes, découper	4
extraire un rôle par client (Reader, Mailable, Chargeable)	7
déplacer l'interface dans le fichier du client (role interface)	6
séparer les capacités qu'une implémentation refuse	2
remplacer un `retry()` par un décorateur	11
découper les props d'un composant au lieu de les rendre optionnelles	9
déclarer une config par module au lieu de `AppConfig`	10
composer deux rôles avec `extends`	6
renommer par le besoin : `Reader`, pas `IService`	2